7006SCN – Machine Learning and Big Data

Distributed Classification of NYC Taxi Trips Using Spark MLlib

Student Name: Gladstone Cletus Austin D'Souza

Student ID: 16676039

Submission Date: 03 March 2026

Project Repository Link:

https://github.com/Glady18D/NYC-Taxi-Spark-ML-Classification/

Tableau Public Dashboard Link:
https://public.tableau.com/app/profile/gladstone.dsouza/vizzes

Dataset Size: 1.35 GB

Word Count: 1,902 words (including table of content, references & appendix)

Table of Contents

7006SCN – Machine Learning and Big Data

1. Abstract
2. Introduction
3. Dataset Overview
    - 3.1 Exploratory Data Analysis (EDA)
    - 3.2 Big Data Challenges
4. Methodology
    - 4.1 Preprocessing Pipeline
    - 4.2 Data Splitting Strategy
    - 4.3 Model Selection and Justification
    - 4.4 Performance Metrics
    - 4.5 Distributed Training and Hyperparameter Tuning
    - 4.6 Computational Considerations
5. Results
    - 5.1 Model Performance Comparison
    - 5.2 Scalability Analysis
6. Discussion
7. Conclusions and Future Work
8. References
9. Appendix – AI Use Declaration

1. Abstract

This project applies distributed machine learning techniques to a large-scale NYC Taxi dataset (1.35GB) to address a binary classification problem: predicting whether a trip qualifies as high-revenue based on trip attributes (distance, duration, passenger count, temporal features). A complete PySpark pipeline was implemented, including distributed ingestion, preprocessing, feature engineering, and MLlib model training. Three distributed algorithms—Logistic Regression, Gradient Boosted Trees, and Random Forest—were evaluated and compared against a single-node scikit-learn Logistic Regression baseline. Performance was assessed using ROC AUC and Precision-Recall AUC. Random Forest achieved the strongest results (ROC AUC 0.91; PR AUC 0.84). Scalability experiments demonstrated strong scaling benefits with diminishing returns due to shuffle overhead. The findings confirm that distributed ML frameworks are essential for scalable urban transportation analytics.

2. Introduction

Urban mobility systems generate large-scale transactional data requiring scalable computational infrastructure. The NYC Taxi dataset represents a realistic big data scenario where single-node machine learning becomes memory-constrained and inefficient.

This study formulates a supervised binary classification task: predicting high-revenue trips. The research question is:

How effectively can distributed machine learning improve predictive performance and scalability on large-scale taxi trip data compared to single-node approaches?

The project covers the full big data lifecycle: ingestion, distributed feature engineering, ML training, scalability analysis, and business-oriented visualisation using Tableau.

3. Dataset Overview

The dataset was sourced from the NYC Taxi & Limousine Commission (TLC) public records portal. It contains over 80 million trip records after preprocessing. The dataset is publicly available for research purposes under open data policies.

The dataset exceeds 1.35GB in size, satisfying the volume characteristic of Big Data (one of the 5Vs), and contains trip-level features including passenger count, trip distance, timestamps, location identifiers, and fare amount. The target variable (`high_revenue_trip`) was engineered using a revenue threshold.

Data was stored in Parquet format to enable columnar compression and Spark-optimised distributed reading. The dataset contains anonymised records, reducing privacy concerns.

3.1 Exploratory Data Analysis

Exploratory analysis revealed:

- Moderate class imbalance between revenue classes
- Strong association between trip duration and revenue
- Temporal variability in pickup hours
- Skewed trip distance distribution

Missing values were validated during ingestion. Temporal features (hour, day-of-week) were extracted, and outlier filtering improved model stability.

Dashboard 1 presents data quality monitoring and trip distribution insights.

3.2 Big Data Challenges

Working with a 1.35GB dataset introduced several technical and architectural challenges associated with distributed data processing.

Spark UI Monitoring Limitation

Spark UI monitoring (localhost:4040) was attempted but restricted due to Colab port forwarding limitations. Optimisation was instead conducted through partition tuning, caching, and execution-time measurement.

Storage and Memory Constraints

The dataset could not be processed entirely in Pandas due to memory limitations on a single-node environment. Attempting to load the full dataset locally resulted in significant memory pressure. Spark's distributed DataFrame architecture enabled partitioned parallel processing across executors, mitigating these constraints and allowing scalable transformation pipelines.

Partitioning and Shuffle Overhead

Improper partitioning initially caused performance bottlenecks during aggregation and grouping operations. Shuffle-heavy transformations increased task coordination costs across partitions. Repartitioning aligned with query patterns significantly reduced shuffle overhead and improved execution efficiency, demonstrating the importance of partition-aware optimisation in distributed systems.

Caching Strategy

Selective use of `persist()` improved performance after computationally expensive transformations, particularly during repeated model training iterations.

However, excessive caching can increase memory pressure. The use of `unpersist()` ensured efficient memory management and prevented resource saturation during experimentation.

Framework Selection

DataFrames were preferred over RDDs due to Spark's Catalyst optimiser, which provides automatic query optimisation and execution planning. This resulted in more efficient execution compared to low-level RDD transformations and reduced the need for manual optimisation.

4. Methodology

4.1 Preprocessing Pipeline

The distributed preprocessing pipeline included:

- Timestamp decomposition
- Trip duration calculation
- Label engineering
- Feature vector assembly via VectorAssembler

SparkSession configuration optimised shuffle partitions and executor memory allocation.

Broadcast joins were not required in this pipeline as transformations were primarily applied to a single large dataset without dimensional enrichment joins. Therefore, standard distributed joins were sufficient.

Spark's lazy evaluation model and immutable DataFrame transformations inherently provide data lineage tracking through logical and physical execution plans, supporting transparency and reproducibility.

4.2 Data Splitting Strategy

An 80/20 stratified split preserved class distribution. Hyperparameter tuning was conducted using Spark MLlib's CrossValidator with parallel execution enabled. Although a temporal split could provide stricter real-world validation, stratified random splitting ensured balanced evaluation.

4.3 Model Selection and Justification

Three distributed MLlib models were implemented:

1. Logistic Regression
2. Gradient Boosted Trees
3. Random Forest

A scikit-learn Logistic Regression model was trained on a 5% sample to serve as a single-node baseline.

Tree-based ensembles were not replicated in scikit-learn due to scalability constraints.

Feature engineering was implemented using Spark ML's built-in transformers (e.g., `VectorAssembler`, `StringIndexer`) within a structured pipeline. A full custom Transformer was not required due to the structured schema and tabular nature of the dataset. However, in production environments, domain-specific transformers could encapsulate business logic such as revenue threshold derivation or advanced temporal feature extraction.

Explicit checkpointing was not required due to controlled DAG depth and manageable computational scale within the experimental environment.

4.4 Performance Metrics

Model performance was evaluated using:

- ROC AUC (overall discrimination ability)
- PR AUC (robustness under class imbalance)

PR AUC was included to ensure reliable evaluation of minority revenue class detection.

While ROC AUC and PR AUC were primary evaluation metrics, operational implications such as resource allocation efficiency and cost-performance trade-offs were considered for business alignment.

4.5 Computational Considerations

Experiments were conducted using Google Colab with a distributed Spark environment configured with multiple executors and tuned shuffle partitions. SparkSession parameters were adjusted to optimise memory allocation and parallel task execution. Caching and repartitioning were applied strategically to minimise shuffle overhead and execution time.

4.6 Hyperparameter Tuning Strategy

Hyperparameter tuning was conducted using Spark MLlib's CrossValidator with parallel execution enabled. Grid search ranges were constrained to maintain computational feasibility while balancing bias and variance.

The final model was serialised using Spark MLlib's `write()` interface to ensure reproducibility.

5. Results

5.1 Model Performance Comparison

Spark MLlib (Full 1.35GB Dataset)

Model	ROC AUC	PR AUC
Logistic Regression	0.81	0.74
Gradient Boosted Tree	0.89	0.82
Random Forest	**0.91**	**0.84**

Random Forest achieved the strongest performance, demonstrating superior nonlinear modelling capacity. The substantial performance gap between ensemble models and linear models suggests the presence of nonlinear interactions between trip features and revenue outcomes.

Scikit-learn Baseline (5% Sample)

Model	ROC AUC	PR AUC
Logistic Regression	0.87	0.79

The scikit-learn baseline achieved competitive results on a reduced dataset but cannot scale to the full 1.35GB dataset.

The Tableau dashboards were built using exported CSV extracts generated from Spark outputs rather than live database connections. This approach ensured efficient rendering and improved performance when visualising aggregated large-scale results.

Level-of-Detail (LOD) expressions were not required, as key aggregations were precomputed within Spark prior to visualisation.

Dashboard 2 visualises distributed versus single-node performance comparison.

5.2 Scalability Analysis

Strong and Weak Scaling Behaviour

Strong scaling was evaluated by increasing executors while maintaining dataset size. Processing time decreased significantly before diminishing returns emerged due to shuffle overhead and coordination costs.

Weak scaling experiments were not conducted due to fixed dataset size constraints; however, executor scaling under constant workload demonstrated diminishing performance returns.

Key findings:

- Distributed execution outperformed sequential processing
- Excessive partitioning introduced coordination overhead
- Ensemble models required greater computational resources
- These results illustrate cost-performance trade-offs in distributed ML systems.

Dashboard 4 presents scalability and cost-performance trade-offs.

6. Discussion

Random Forest outperformed Logistic Regression due to its ability to capture nonlinear interactions between trip features and revenue patterns. Gradient Boosted Trees performed competitively but required careful tuning.

Although the scikit-learn baseline achieved higher ROC AUC (0.87) than Spark Logistic Regression (0.81), this comparison reflects sampling differences. Spark's evaluation was conducted on the full 1.35GB dataset, providing a more realistic representation of large-scale performance.

PR AUC results further confirmed ensemble superiority, with Random Forest achieving 0.84 compared to Logistic Regression's 0.74.

An additional observation relates to the trade-off between predictive performance and computational cost. While Random Forest achieved the highest ROC AUC (0.91) and PR AUC (0.84), it required greater training time compared to Logistic Regression. This highlights a practical deployment consideration: marginal performance improvements must be evaluated against computational cost and infrastructure scalability. In production environments, cost-performance optimisation is often as important as predictive accuracy.

The Tableau dashboards prioritised narrative clarity over complex interactivity. While parameter controls and action filters were possible, annotated insights were used to guide stakeholder interpretation.

7. Conclusions and Future Work

This study demonstrates that distributed Spark MLlib effectively handles large-scale taxi datasets while maintaining strong predictive performance. Random Forest achieved the highest ROC AUC (0.91) and PR AUC (0.84).

The scikit-learn baseline validated model behaviour but highlighted single-node scalability limitations.

Future work may include:

- Temporal validation strategies
- Real-time Spark Streaming integration
- SHAP-based interpretability
- Larger hyperparameter search
- Deployment on managed cluster infrastructure

Distributed ML frameworks are essential for scalable urban mobility analytics.

Dashboards were published on Tableau Public, enabling stakeholder accessibility and export functionality for reporting purposes.

8. References

(APA 7th Edition Format)

Apache Software Foundation. (2024). *Apache Spark™ – Unified analytics engine for large-scale data processing*. https://spark.apache.org

Laney, D. (2001). *3D data management: Controlling data volume, velocity and variety*. META Group.

Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M., Zadeh, R., Zaharia, M., & Talwalkar, A. (2016). MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research, 17*(34), 1–7.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation* (pp. 15–28). USENIX Association.

9. Appendix – AI Use Declaration

In accordance with the "Amber" rating for AI tool use, generative AI was used to assist with structural organisation, grammar refinement, and explanation of distributed machine learning concepts. All data processing, modelling implementation, experimentation, evaluation, and interpretation were independently conducted and verified by the author.

Tool	**How used in this assignment**
ChatGPT	Structural refinement of report sections, clarification of distributed ML concepts, and assistance with grammar improvements.
Grammarly	Grammar checking, sentence clarity enhancement, and formatting consistency.